

LLMsVerifier

LLMsVerifier

Верификуј. Надгледај. Оптимизуј.

Сажетак

LLMsVerifier је платформа пословне класе за верификацију, надгледање и оптимизацију великих језичких модела (LLM) код различитих провајдера, изграђена на обавезном тесту верификације „*Видиш ли мој код?*“ како би само модели за које је доказано да заиста функционишу икада били означени као употребљиви или извезани.

Кратак опис

Платформа Go која верификује, тестира перформансе, надгледа и оптимизује LLM-ове код више провајдера. Сваки модел мора проћи обавезни тест видљивости кода пре употребе; затим се проверавају латенција, стримовање, позиви функција, визуелна обрада и уградња, а извозе се само верификоване конфигурације за алате AI и CLI.

Детаљан опис

LLMsVerifier је свеобухватна платформа за верификацију, надгледање и оптимизацију перформанси LLM модела код различитих провајдера. Њен основни принцип је *обавезна верификација*, и у томе је непопустљива: пре него што се било који модел означи као употребљив — или дозволи у извозну конфигурацију — мора изричито да прође тест „*Видиш ли мој код?*“, који шаље стварне ХТТП захтеве провајдеру и анализира одговор како би утврдио стварно разумевање, а не само веродостојан одјек. Модел који не може недвосмислено да види и разуме ваш унос једноставно никада не добија ознаку „употребљив“. Након те провере, Верификациони мотор спроводи комплетну серију тестова способности —

постојање, респонсивност, латенција, стримовање, позиви функција, визуелна обрада, ембеддингс — а Извештајни мотор резултате претвара у markdown и JSON извештаје на основу којих можете предузимати акције.

Систем је модуларан и вођен догађајима, нудећи CLI, TUI, веб и REST API интерфејсе преко језгра које чине Верификациони мотор, Извештајни мотор и Менаџер конфигурација, и не зауставља се на верификацији. Напредни слојеви додају Супервизор/Радник образац за декомпозицију задатака покретаних LLM моделима, управљање контекстом помоћу клизећег прозора и LLM сумаризације како веома дуге сесије не би доживеле колапс, чување контролних тачака у облаку и систем за преузимање у случају отказа са прекидачима струјног кола и рутирањем заснованим на латенцији. Околна инфраструктура је прилагођена продукцији: магистрала догађаја типа пуб/суб, распоред црон послова, детекција цена и ограничења, вектор база података за RAG, и систем за извоз. Посебна конвенција брендирања додаје суфикс (*лмсвд*) сваком генерисаном провајдеру/моделу, тако да се верификовани излаз може препознати на први поглед и никада не може бити помешан са непровереним — а само верификовани модели се икада уписују у извезене конфигурације за AI CLI алате попут OpenCode, Crush и Claude Цоде. Долази са оперативним алатима које тимови заиста користе у продукцији: Docker/Kubernetes/Helm имплементација, Prometheus/Grafana надгледање, LDAP/SSO и SQLCipher енкриптовано складиштење.

Зашто смо је изградили

Зато што је провера само на основу конфигурације непоуздана — API кључ може истећи, модел може бити застарео, а конфигурациони фајл вам не говори ништа о стварној латенцији, стварним грешкама или томе да ли модел заиста види и разуме ваш унос. LLMsVerifier замењује *„налази се у конфигурацији, дакле мора да ради“* доказом: само модели који недвосмислено одговарају исправно бивају означени као употребљиви и извозе се.

Зашто је ово револуционарно

Чини LLM флоте *поузданим* — реч која се ретко заслужује у свемиру подешавања која лажу изостављањем. Уместо да се тимови надају да ће конфигурисани модел радити, добијају строго спроведену, тестабилну гаранцију да је сваки модел у употреби прошао стварну верификацију, уз мониторинг, преусмеравање у случају грешке и извоз само верификованих модела који затварају круг од доказа до продукције. У оквиру

Helix екосистема постаје једини извор истине за LLM моделе, провајдере и метаподатке верификације: други сервиси (међу њима и HelixTranslate) усмеравају саобраћај према њему, па цела платформа наслеђује један поштен одговор на питање „*који модели заиста сада раде?*“ уместо да сваки тим одржава сопствено оптимистично нагађање.

Шта је иновативно

- **Обавезна верификација „Видиш ли мој код?“** — стварна, ХТТП-ослоњена провера разумевања коју модел мора да прође пре него што уопште постане употребљив; потписна карактеристика производа и разлог што ништа непроверено не проклизне.
- **Извоз конфигурације само верификованих модела** — генерисане конфигурације за AI CLI алате садрже *само* моделе који су прошли верификацију, тако да конфигурација коју шаљете не може тихо да реинтродукује покварен модел.
- **Систем суфикса брендирања (llmsvd)** — сваки генерисани провајдер/модел носи уочљив суфикс, чинећи верификовано порекло видљивим свуда где се излаз шири.
- **Детекција могућности** на многим CLI агентима и провајдерима — отискује типове стриминга (SSE, WebSocket, JSONL, EventStream), компресију и понашање кеширања уместо да их претпоставља.
- **Отпормо преусмеравање** — прекидачи струјног кола, рутирање засновано на латенцији које преусмерава када време до првог токена пређе праг, пробе здравља и пондерисана расподела саобраћаја одржавају флоту реакљивом када поједини провајдери закажу.
- **Дуготрајна аутономија** — образац декомпозиције Супервизор/Радник уз контролне тачке и интеграцију меморије подржава продужене сесије које би иначе исцрпле контекст.
- **Интеграција са RAG / вектор-ДБ** за побољшање контекста заснованог на чињеницама.

Највећи технички изазови и како смо их решили

- **Доказивање да модел заиста ради, а не само да је конфигурисан.** Цела поента, и најтежи део. Решено обавезним тестом видљивости кода који прави стварне API позиве и анализира одговоре на потврдно разумевање, подржано широким скупом тестова могућности — а затим одбијањем извоза свега што није прошло, тако да продукцију не контролише конфигурација, већ доказ.

- **Поузданост на многим нестабилним провајдерима трећих страна.** Решено оркестром за преусмеравање који нестабилност провајдера третира као нормалан случај: прекидачи струјног кола означавају провајдера као деградираног након N грешака у M секунди, рутирање засновано на латенцији скреће са спорих крајњих тачака, периодичне пробе здравља проверавају опоравак, а пондерисано рутирање балансира између економичних и премиум модела.
- **Одржавање веома дугих, аутономних сесија.** Решено обрасцем декомпозиције Супервизор/Радник који дели велики посао на управљиве делове, периодичним контролним тачкама у цлоуд складишту како би напредак преживео прекиде, и слојевитим управљањем контекстом (клизећи прозор + сумаризација LLM + RAG) како модел не би изгубио нит а да се не удави у токенима.
- **Ширење провајдера.** Решено скривањем многих Go адаптера за поједине провајдере иза једног заједничког интерфејса, уз централно набрајање стварних крајњих тачака — тако да додавање провајдера представља изоловану промену, а не талас који се шири кроз кодну базу.

Технолошки стек

- **Go** — изабран као основни програмски језик због подршке конкурентности; покреће вишедретвени Верификациони мотор који може истовремено да испитује више модела, као и околне сервисе.
- **Gin** — изабран као REST API сервер, који подржава JWT аутентификацију, ограничење броја захтева и WebSocket/SSE крајње тачке.
- **SQLite + SQLCipher** — изабрани за уграђено складиштење са енкрипцијом на нивоу базе података, јер су верификациони подаци (кључеви, резултати) осетљиви и подразумевано треба да буду шифровани у мировању.
- **Redis** — изабран као слој за кеширање како би брзе провере и претраге метаподатака остале брзе.
- **RabbitMQ + Kafka** — изабрани за покретање архитектуре вођене догађајима: размену порука и стримовање које одваја произвођаче од потрошача на нивоу платформе.
- **gRPC + Protocol Buffers** — изабрани за строго типизiranу комуникацију између сервиса и пренос догађаја између компоненти.
- **QUIC / HTTP-3 (quic-go)** — изабрани за подршку модерном транспорту (у документацији репозиторијума наводи се да је доступност HTTP/3 провајдера ограничена — то је понуђена могућност, а не универзална тврдња).
- **JWT + LDAP/NTLM** — изабрани за предузетничку аутентификацију како би се платформа уклопила у

постојећи корпоративни идентитет (у документацији се наводе SSO/SAML/OIDC).

- **Viper (конфигурација), Logrus (евиденција), Brotli/цомпресс (компресија)** — оперативна инфраструктура: флексибилна конфигурација, структурирани записи и компресија корисног оптерећења.
- **Angular** — изабран за веб апликацију у виду једне странице, визуелни улаз у верификацију и надзор.
- **Python + JavaScript SDK-ови** — изабрани како би тимовима клијената пружили првокласан приступ, документован путем OpenAPI/Swagger-а.
- **Docker, Kubernetes, Helm** — изабрани за продукцијско постављање са праћењем здравља система и аутоматским скалирањем, како би флота за верификацију скалирала попут сваког модерног сервиса.
- **Prometheus + Grafana** — изабрани за метрику и контролне табле, чинећи здравље саме платформе једнако уочљивим као и моделе које надгледа.
- **Testify (Go) + ноде -тест/јсдом (веб)** — изабрани за вишеслојно тестирање Go језгра и веб интерфејса.

Статус и напомене о искрености

- **Статус: бета.** Изворни код Go имплементира стварну ХТТП верификацију (један застарели документ који верификацију описује као искључиво конфигурациону представља аспирацију и застарео је — ауторитативан је код).
- **Лиценца: треба утврдити.** У РЕАДМЕ фајлу наводи се МИТ, док Доцкерфиле ознака наводи Апацхе-2.0 — питање треба разрешити пре објављивања.
- Број провајдера: у РЕАДМЕ фајлу стоји „12 адаптера”, али директоријум провајдера наводи око 26 — третирајте као „12+ / више у развоју”. Постоји много фајлова са статусом „ФИНАЛНО/КОМПЛЕТНО” који представљају аспирацију; ауторитативни су код, документација и go.mod.
- Репозиторијум се налази у организацији vasic-digital, али функционално представља слој поверења Helix LLM инфраструктурног кластера.

Приоритетни ниво: Helix-пример (LLM инфраструктурни кластер; једини извор истине за LLM/провајдер/верификационе метаподатке). Рангира се иза HelixTrack.